

Welcome to AdManager Pro - Easy Setup

This powerful tool simplifies the integration of various ad networks into your Unity projects, allowing you to easily manage and optimize your ad revenue. With built-in support, our asset provides everything you need for effective ad monetization all in one place.

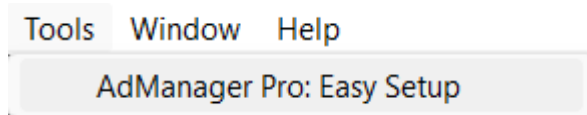
Enjoy easy setup, cross-platform compatibility, and documentation to help you get started quickly and efficiently.

Demo App

Download Android APK-file AdManager Pro: Easy Setup: [Link](#)

Admob Setup

Select **AdManager Pro - Easy Setup** from the **Tools** tab.



In the window that opens, follow the setup steps.

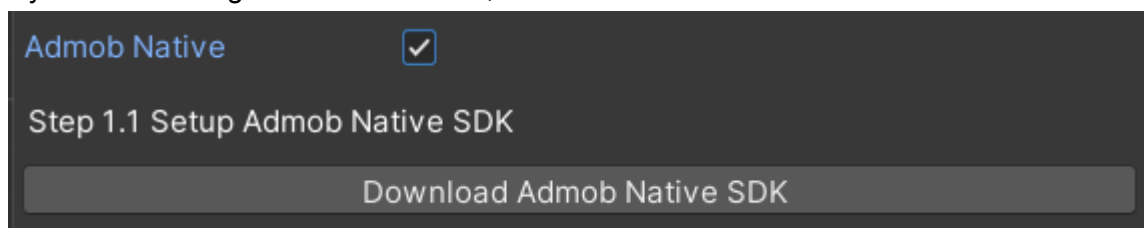
Download SDK

Press the **Download Admob SDK** button or download SDK from [link](#)

Import the SDK into your Unity project.

Copy

If you will be using AdMob native ads, check this box and download the SDK.



Press the **Download Admob Native SDK** button or download SDK from [link](#)

Import the SDK into your Unity project.

Test Mode

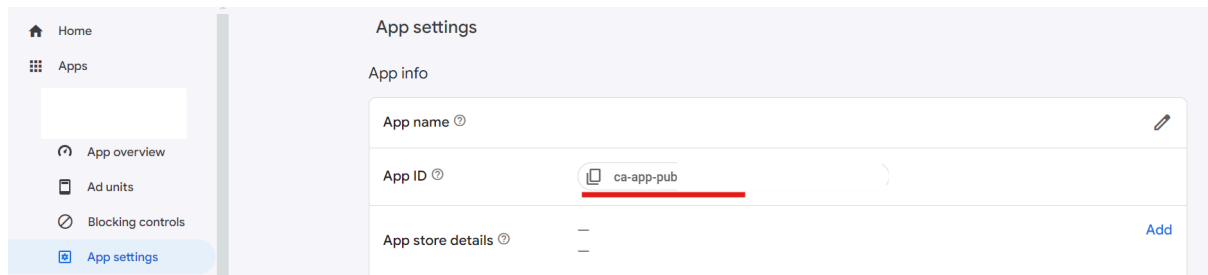
When Test Mode is enabled, the default test IDs from AdMob will be used instead of the ones provided in this window. Always enable Test Mode to verify that your integration is working correctly.

Remember to disable Test Mode before releasing your app.

App ID

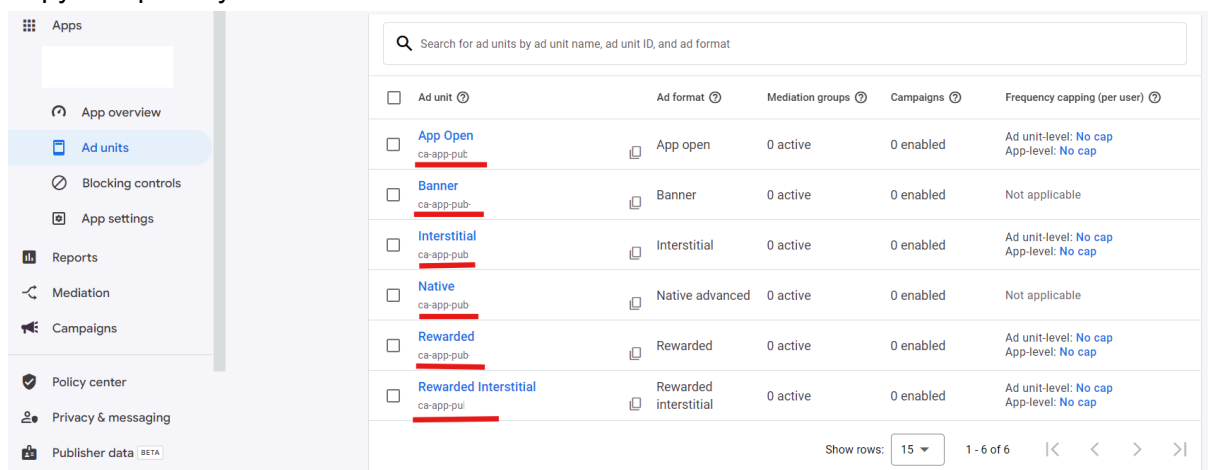


Copy and paste the app ID from the App Settings section of your Admob dashboard.



AD IDs

Copy and paste your Ad units from the Admob dashboard



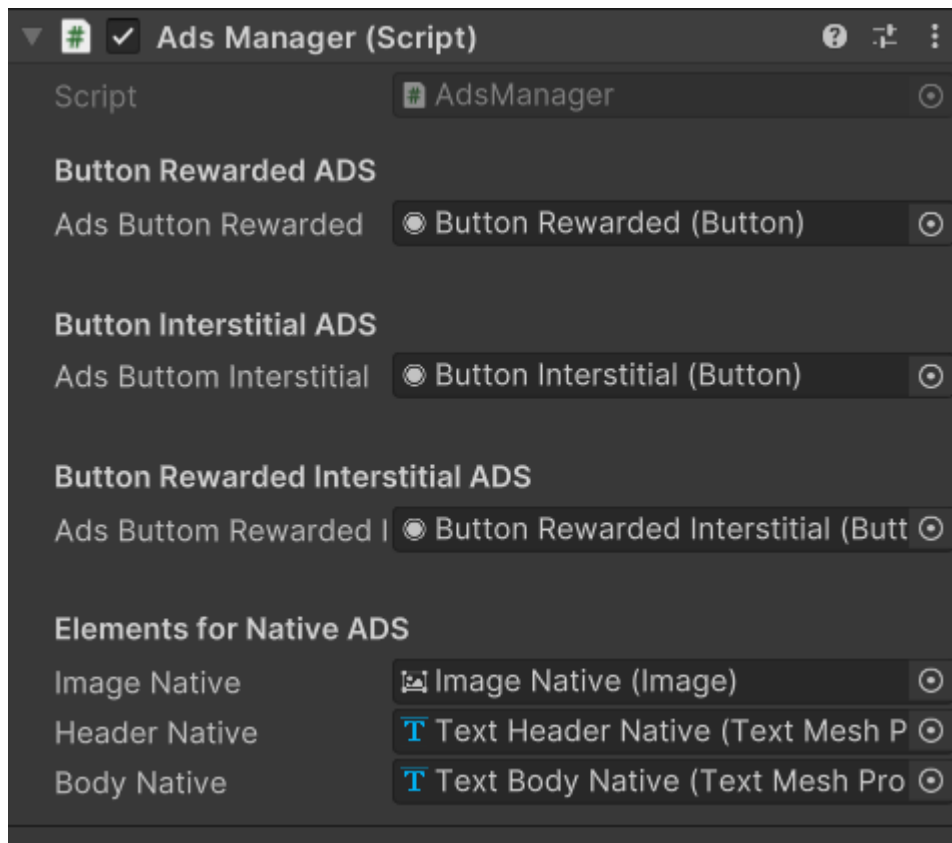
You don't have to fill in all the fields. If you only want to use Rewarded ads, simply copy and paste that ID. This won't affect the functionality of the asset in any way.

Save

Save all your data by clicking the Save button.

Start

If you want to explore an example, you can click the "Example Scene" button and take a look at the AdsManager object.



IMPORTANT! To use the asset, make sure to include this line in your script:

Copy

```
using AdManagerPro;
```

Banner

If you want to use an adaptive banner, make sure to check the option in the asset settings.



Create a request to load the ad.

You should call this part of the code once at the beginning.

Copy

```
using UnityEngine;
```

```
using AdManagerPro;
```

```
public class Test : MonoBehaviour {
```

```
    private void Start()
```

```
    {
```

```

        // Requests to download advertisements

        BannerAdManager.RequestAdBanner();

    }

}

```

IMPORTANT! By default, the banner position is set to the bottom. To change the banner position, you need to add `AdPosition` to the request and select the desired position.

Copy

```

BannerAdManager.RequestAdBanner(AdPosition.Top);

```

SHOW, HIDE, or DESTROY

You can SHOW, HIDE, or DESTROY the banner using the following code snippets:

Show Banner:

Copy

```

BannerAdManager.ShowAdBanner();

```

Hide Banner:

Copy

```

BannerAdManager.HideAdBanner();

```

Destroy Banner:

Copy

```

BannerAdManager.DestroyAdBanner();

```

If you need to check the availability/loading status of the ad, include the following code snippet:

```

using UnityEngine;

using AdManagerPro;

public class Test : MonoBehaviour

{

    private void Start()

    {

```

```
if(BannerAdManager.isAdmobBannerReady){  
    Debug.Log("The ad is available.");  
}  
}  
}
```

Interstitial

Create a request to load the ad.

You should call this part of the code once at the beginning.

Copy

```
using UnityEngine;
```

```
using AdManagerPro;
```

```
public class Test : MonoBehaviour
```

```
{
```

```
    private void Start()
```

```
    {
```

```
        // Requests to download advertisements
```

```
        InterstitialAdManager.RequestAdInterstitial();
```

```
    }
```

```
}
```

Show Interstitial ADS

To show interstitial ads, you can use the following code snippet:

Copy

```
InterstitialAdManager.ShowAdInterstitial();
```

This method will display an interstitial ad, typically used for full-screen ads between content or levels in your game.

If you need to check the availability/loading status of the ad, include the following code snippet:

```
using UnityEngine;

using AdManagerPro;

public class Test : MonoBehaviour

{

    private void Start()

    {

        if(InterstitialAdManager.isAdmobInterstitialReady){

            Debug.Log("The ad is available.");

        }

    }

}
```

Rewarded Interstitial

Create a request to load the ad.

You should call this part of the code once at the beginning.

Copy

```
using UnityEngine;

using AdManagerPro;

public class Test : MonoBehaviour

{

    private void Start()

    {

        // Requests to download advertisements
```

```
RewardedInterstitialAdManager.RequestRewardedInterstitialAd();  
  
}  
  
}
```

Show Rewarded Interstitial ADS

To show rewarded interstitial ads, you can use the following code snippet:

Copy

```
RewardedInterstitialAdManager.ShowRewardedInterstitialAd(OnAdWatched);
```

OnAdWatched is a delegate used as a callback after a rewarded ad has either been shown and watched or not shown. This delegate takes a single parameter of type `bool`, which indicates whether the ad was watched.

Implement a method for the player to receive a reward.

Copy

```
private void OnAdWatched(bool adWatched)  
  
{  
  
    if (adWatched)  
  
    {  
  
        Debug.Log("Ad watched, reward added!");  
  
    }  
  
    else  
  
    {  
  
        Debug.Log("Ad was not watched.");  
  
    }  
  
}
```

If you need to check the availability/loading status of the ad, include the following code snippet:

Copy

```
using UnityEngine;
```

```
using AdManagerPro;
```

```

public class Test : MonoBehaviour

{

    private void Start()

    {

        if(RewardedInterstitialAdManager.isAdmobRewardedInterstitialReady){

            Debug.Log("The ad is available.");

        }

    }

}

```

Rewarded

Create a request to load the ad.

You should call this part of the code once at the beginning.

Copy

```
using UnityEngine;
```

```
using AdManagerPro;
```

```

public class Test : MonoBehaviour

{

    private void Start()

    {

        // Requests to download advertisements

        RewardedAdManager.RequestRewardedAd();

    }

}

```

Show Rewarded ADS

To show rewarded ads, you can use the following code snippet:

Copy

```
RewardedAdManager.ShowRewardedAd(OnAdWatched);
```

OnAdWatched is a delegate used as a callback after a rewarded ad has either been shown and watched or not shown. This delegate takes a single parameter of type `bool`, which indicates whether the ad was watched.

Implement a method for the player to receive a reward.

Copy

```
private void OnAdWatched(bool adWatched)
```

```
{  
  
    if (adWatched)  
  
    {  
  
        Debug.Log("Ad watched, reward added!");  
  
    }  
  
    else  
  
    {  
  
        Debug.Log("Ad was not watched.");  
  
    }  
  
}
```

If you need to check the availability/loading status of the ad, include the following code snippet:

Copy

```
using UnityEngine;
```

```
using AdManagerPro;
```

```
public class Test : MonoBehaviour
```

```
{  
  
    private void Start()  
  
    {
```

```

        if(RewardedAdManager.isAdmobRewardedReady){

            Debug.Log("The ad is available.");

        }

    }

}

```

App Open

Create a request to load the ad.

You should call this part of the code once at the beginning.

Copy

```
using UnityEngine;
```

```
using AdManagerPro;
```

```
public class Test : MonoBehaviour
```

```
{
```

```
    private void Start()
```

```
    {
```

```
        // Requests to download advertisements
```

```
        AppOpenAdManager.LoadAppOpenAd();
```

```
    }
```

```
}
```

Show App Open

To trigger an App Open ad, you need to perform a check when the application is minimized by the player, as shown in the example below.

Copy

```
private void OnApplicationPause(bool pause)
```

```
{
```

```

        if (pause == false)

        {

            AppOpenAdManager.ShowAppOpenAd();

        }

    }

```

If you need to check the availability/loading status of the ad, include the following code snippet:

Copy

```
using UnityEngine;
```

```
using AdManagerPro;
```

```
public class Test : MonoBehaviour
```

```

{

    private void Start()

    {

        if(AppOpenAdManager.isAdmobAppOpenReady){

            Debug.Log("The ad is available.");

        }

    }

}

```

Native

Set up the images and text fields.

Create Image and 2 Text or TextMeshPro fields.

```

1 reference
[SerializeField] Image imageNative;
1 reference | 1 reference
[SerializeField] TMP_Text headerNative, bodyNative;

```

Assign the required values to the fields, and only then make the request.

Copy

```
using UnityEngine;
```

```
using AdManagerPro;
```

```
using UnityEngine.UI;
```

```
using TMPro;
```

```
public class Test : MonoBehaviour
```

```
{
```

```
    [SerializeField] Image imageNative;
```

```
    [SerializeField] TMP_Text headerNative, bodyNative;
```

```
    private void Start()
```

```
    {
```

```
        NativeAdManager.adImage = imageNative;
```

```
        NativeAdManager.adBodyText = bodyNative;
```

```
        NativeAdManager.adHeadlineText = headerNative;
```

```
        // Request to download native ad
```

```
        NativeAdManager.RequestNativeAd();
```

```
    }
```

```
}
```

Availability of Native Ads

If you need to check the availability/loading status of the ad, include the following code snippet:

Copy

```
using UnityEngine;
```

```
using AdManagerPro;

public class Test : MonoBehaviour

{
    private void Start()
    {
        if(AppOpenAdManager.isAdmobAppOpenReady){
            Debug.Log("The ad is available.");
        }
    }
}
```

IMPORTANT! Native ads are loaded only within the actual application, so you won't see them in the Unity editor. Consequently, checking for ad availability will always return "false" in the editor. Make sure to test the ads by creating a test version of your app.